



Tsinghua University

Modern Computer Architecture

Fall 2025

Lecture 12. Warehouse-Scale Computers

Yongpan Liu

ypliu@tsinghua.edu.cn

2025-Dec-17

Introduction

- Warehouse-scale computer (WSC)
 - Provides Internet services
 - Search, social networking, online maps, video sharing, online shopping, email, cloud computing, etc.
- Differences with HPC “clusters”:
 - Clusters have higher performance processors and network
 - Clusters emphasize thread-level parallelism, WSCs emphasize request-level parallelism
- Differences with datacenters:
 - Datacenters consolidate different machines and software into one location
 - Datacenters emphasize virtual machines and hardware heterogeneity in order to serve varied customers

WSC Design Factors (I)

- Cost-performance (i.e., work done/\$)
 - A WSC infrastructure can cost \$150 M
 - A 10% reduction in capital cost saves \$15 M
 - Small savings add up.
- Energy efficiency
 - Affects power distribution and cooling
 - Work done per joule
- Dependability via redundancy:
 - At least 99.99% availability: down less than one hour/year
 - Multiple WSCs
- Network I/O: Inter and intra WSC
- Interactive and batch processing workloads (e.g., MapReduce)

WSC Design Factors (II)



- Energy, power distribution, and cooling: more than 30% of cost over 10 years.
- Scale and its opportunities and problems
 - Can afford to build customized systems since WSC require volume purchase
 - Flipside: failures.

Outages and Anomalies

Approx. number events in 1st year	Cause	Consequence
1 or 2	Power utility failures	Lose power to whole WSC; doesn't bring down WSC if UPS and generators work (generators work about 99% of time).
4	Cluster upgrades	Planned outage to upgrade infrastructure, many times for evolving networking needs such as recabling, to switch firmware upgrades, and so on. There are about nine planned cluster outages for every unplanned outage.
1000s	Hard-drive failures	2%–10% annual disk failure rate (Pinheiro et al., 2007)
	Slow disks	Still operate, but run $10\times$ to $20\times$ more slowly
	Bad memories	One uncorrectable DRAM error per year (Schroeder et al., 2009)
	Misconfigured machines	Configuration led to $\sim 30\%$ of service disruptions (Barroso and Hölzle, 2009)
	Flaky machines	1% of servers reboot more than once a week (Barroso and Hölzle, 2009)
5000	Individual server crashes	Machine reboot; typically takes about 5 min (caused by problems in software or hardware).

Figure 6.1 List of outages and anomalies with the approximate frequencies of occurrences in the first year of a new cluster of 2400 servers. We label what Google calls a cluster an *array*; see Figure 6.5. Based on Barroso, L.A., 2010.

[Computer Architecture, Sixth Edition: A Quantitative Approach]

Outages and Anomalies Example

Calculate the availability of a service running the 2400 servers of the previous table.

Assume:

- service cannot tolerate software or hardware failures (unrealistic)
- Reboot time= 5 minutes
- Hardware repair time= 1 hour
- Ignore slow disks
- All events have equal probability

Hours outage= $(1+4+250+250+250) \times 1\text{hr} + (250+5000)/12$

Availability= $(365 \times 24 - 1193) / (365 \times 24) = 86\%$

Prgrm'g Models and Workloads

- Batch processing framework: MapReduce
 - **Map:** applies a programmer-supplied function to each logical input record
 - Runs on thousands of computers
 - Provides new set of key-value pairs as intermediate values
- **Reduce:** collapses values using another programmer-supplied function

Popularity of MapReduce at Google

	Aug-04	Mar-06	Sep-07	Sep-09
Number of MapReduce jobs	29,000	171,000	2,217,000	3,467,000
Average completion time (seconds)	634	874	395	475
Server years used	217	2002	11,081	25,562
Input data read (terabytes)	3288	52,254	403,152	544,130
Intermediate data (terabytes)	758	6743	34,774	90,120
Output data written (terabytes)	193	2970	14,018	57,520
Average number of servers per job	157	268	394	488

Figure 6.2 Annual MapReduce usage at Google over time. Over five years the number of MapReduce jobs increased by a factor of 100 and the average number of servers per job increased by a factor of 3. In the last two years the increases were factors of 1.6 and 1.2, respectively [Dean 2009]. Figure 6.16 on page 459 estimates that running the 2009 workload on Amazon's cloud computing service EC2 would cost \$133M.

Big Data: Use for MapReduce

- Size of the data becomes part
- Complex structured and unstru
- Heterogeneous and continuou
- Data generated from a variety of scientific instruments, records
- Applications: intelligence, heal
- prevention, crime prevention, e



Big Data: Use for MapReduce

- Size of the data becomes part of the computational solution.
- Complex structured and unstructured data.
- Heterogeneous and continuously generated data.
- Data generated from a variety of sources: sensors, cameras, scientific instruments, records of transactions, etc.
- Applications: intelligence, health-care, business analytics, disease prevention, crime prevention, etc.

Big Data

- The federal government has announced a \$200 million/yr Big Data initiative for research and development into technologies to “access, store, visualize, massive and complicated data sets.”
- Paradigm for BigData computation:
 - MapReduce (introduced by Google): data and processing parallelism
 - Hadoop: open source implementation of MapReduce.
 - Spark: memory access supported

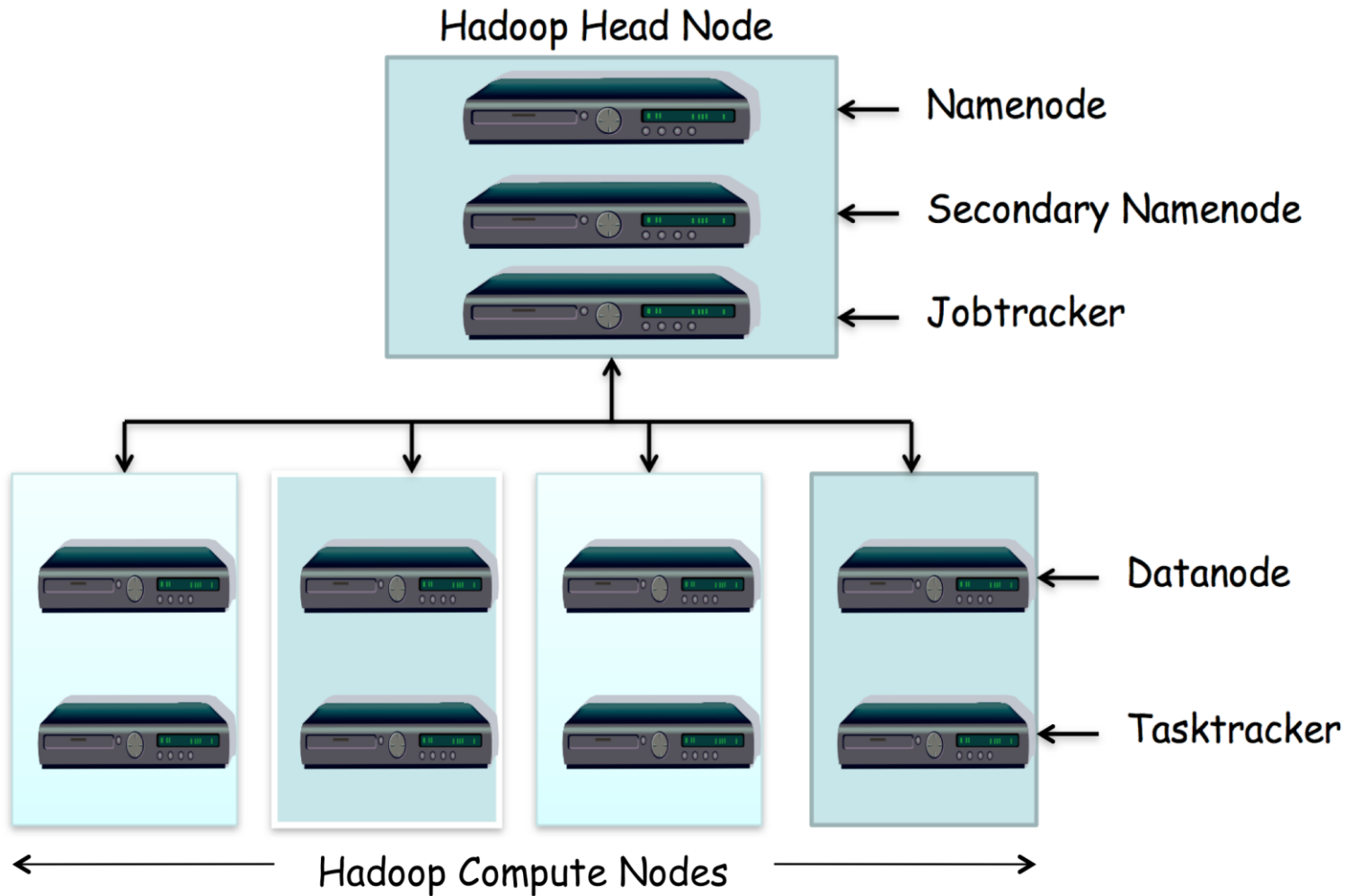
Prgrm'g Models and Workloads

- **MapReduce runtime environment schedules map and reduce task to WSC nodes**
- **Availability:**
 - Use replicas of data across different servers
 - Use relaxed consistency:
 - No need for all replicas to always agree
- **Workload demands**
 - Often vary considerably

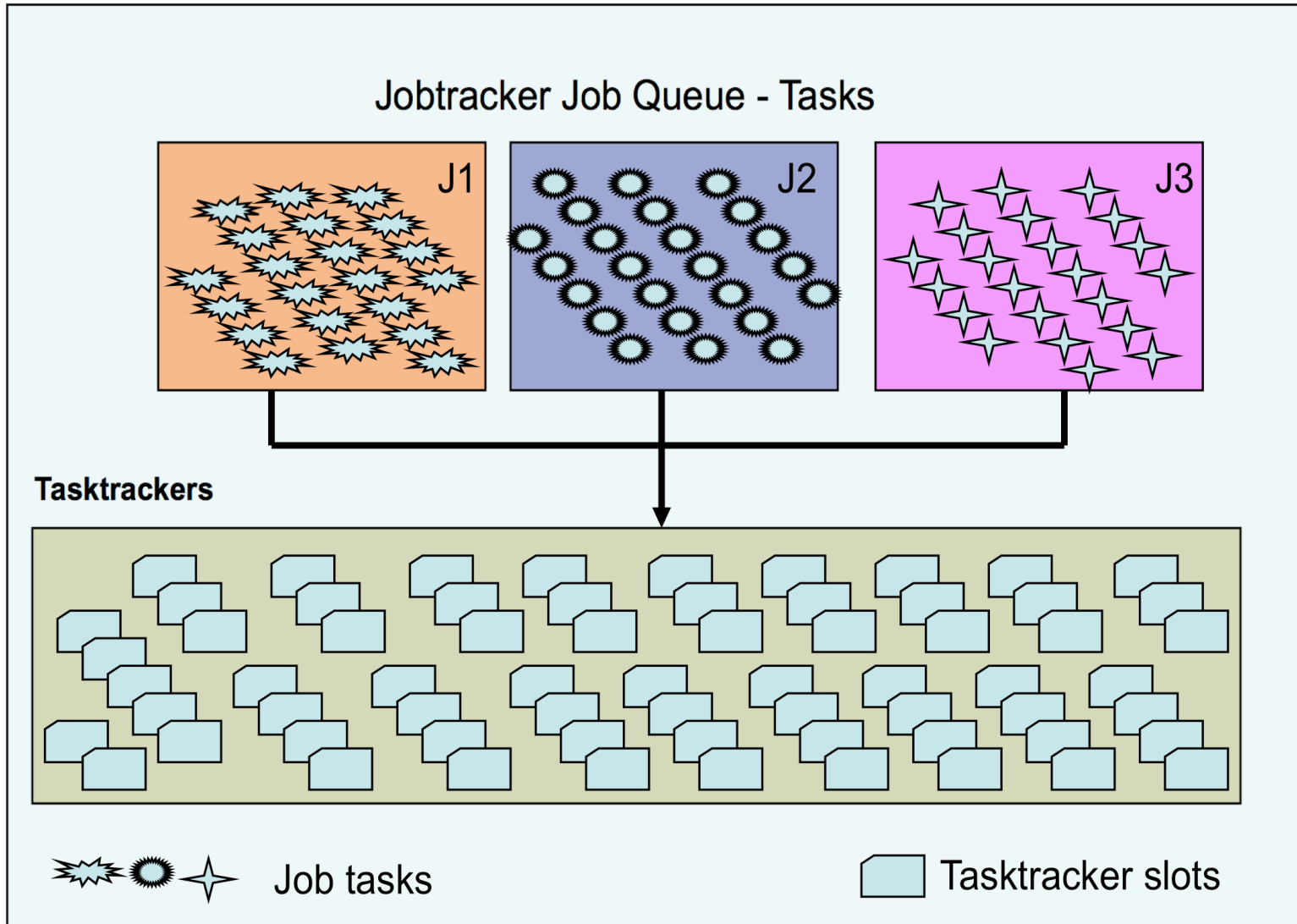
Hadoop: MapReduce Open Source

- Hadoop jobs consist of two types of tasks:
 - Map and Reduce tasks.
 - Reduce tasks can only start when all map tasks have finished.
- Hadoop cluster:
 - Compute nodes: where map and reduce tasks execute
 - Each compute node has a number of slots for the execution of map and reduce tasks.
 - Each compute node has a TaskTracker daemon that monitors the execution of map and reduce tasks.

Hadoop Distributed File System (HDFS) / MapReduce Framework



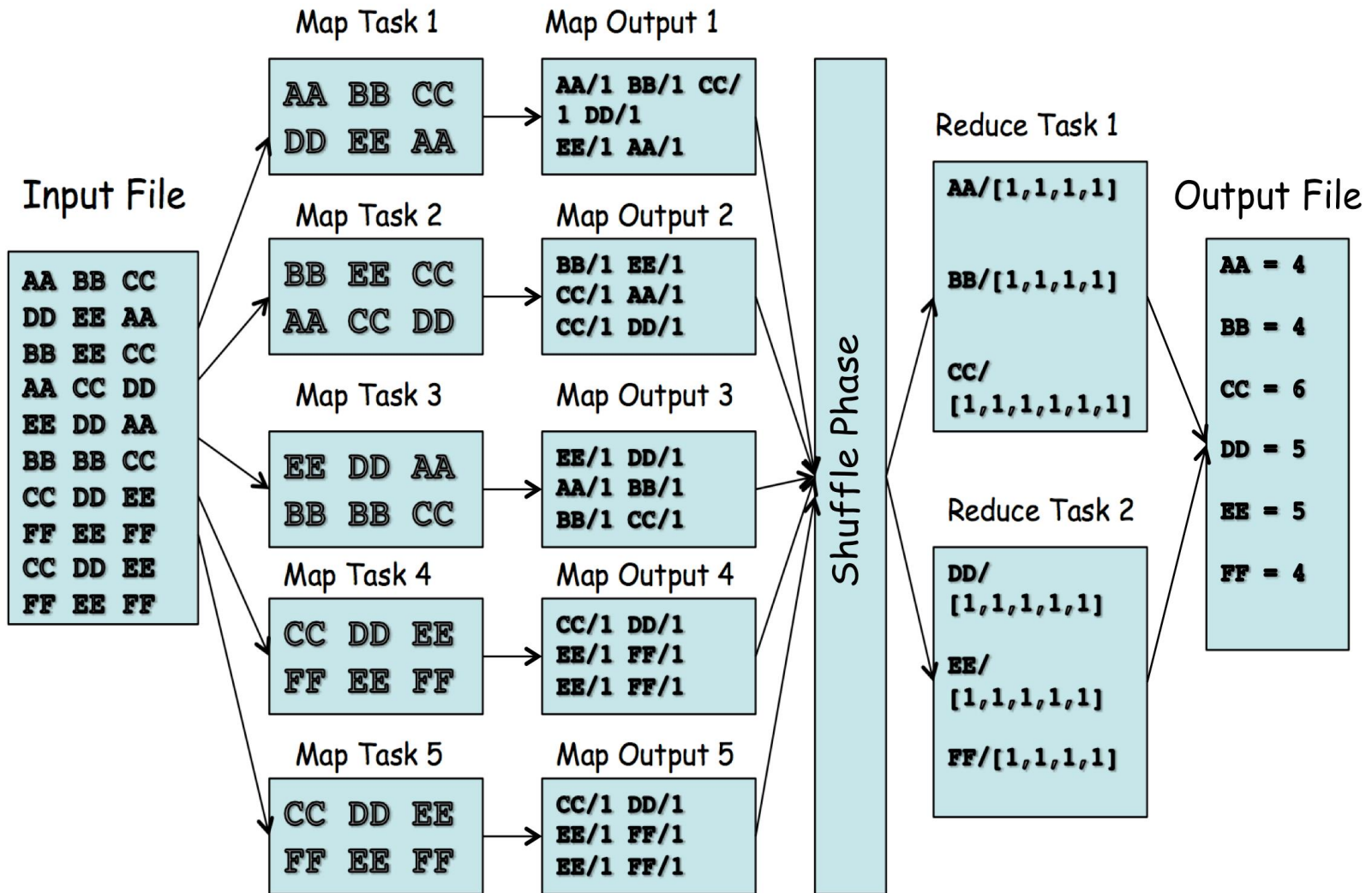
Hadoop Jobtracker/Tasktracker



WordCount Example

- Goal: count the number of occurrences of each word in a large file.
- Approach:
 - File is broken down into a large number of chunks.
 - Each map task takes an input line from the file and breaks it into words and emits a (key, value) pair consisting of the word and a value of “1”.
 - The reduce tasks add the number of “1”s for each word and output the word and the sum.

Map/Shuffle/Reduce Data Flow - Wordcount



WordCount Example: Map task

WordCount Map Method

```
IntWritable one = new IntWritable(1);
Text word = new Text();
public void map (Object key, Text value,
Context context) throws .... {
    StringTokenizer itr =
        new StringTokenizer (value.toString());
    while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
    }
}
```

WordCount Example: Reduce task

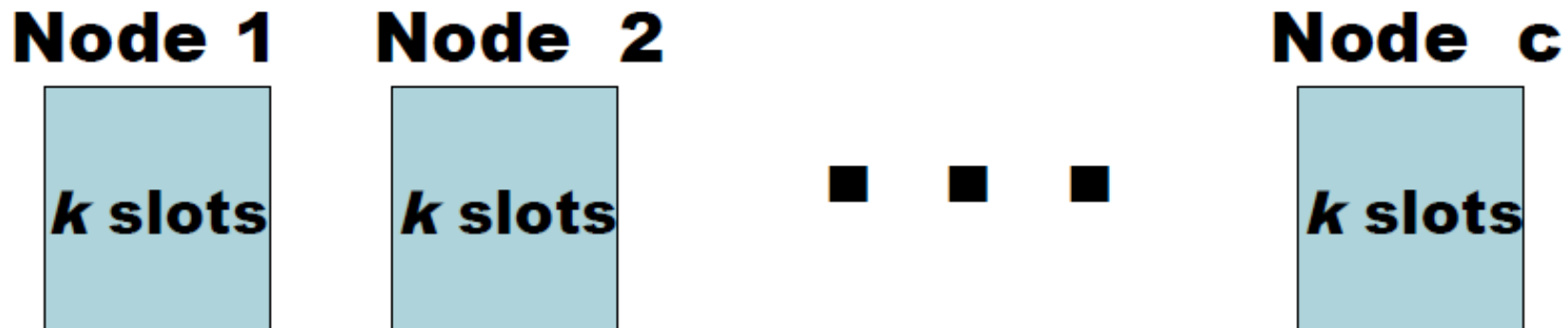
WordCount Reduce Method

```
IntWritable result = new IntWritable();
public void reduce(Text key,
    Iterable<IntWritable> values,
    Context context) throws ..... {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
```

MapReduce Model

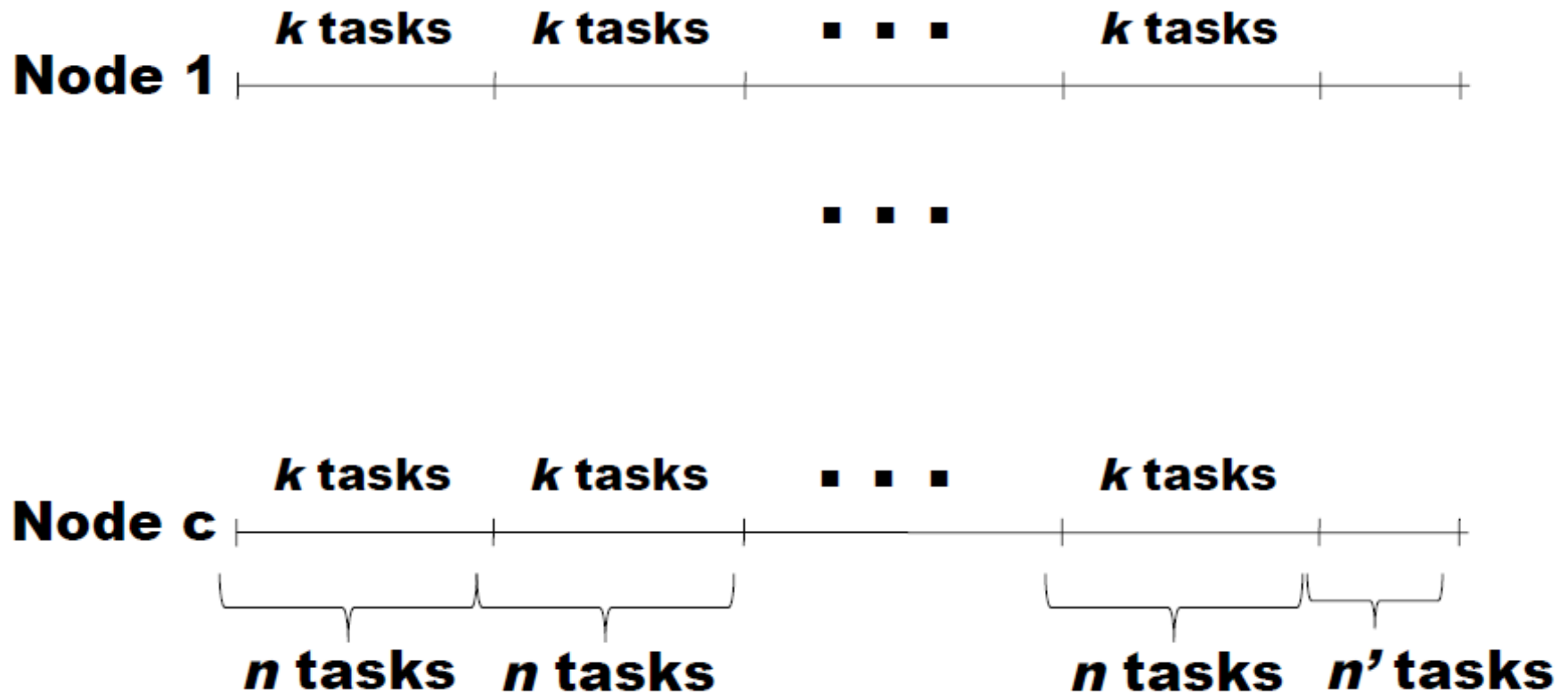
- M : number of map tasks of a job
- n : total number of map slots in the worker nodes
- c : number of compute nodes
- Assume that the n slots are equally distributed among the c compute nodes: $n = k \times c$, where k is the number of map slots per node.

MapReduce Model



$$***n = k \times c \text{ concurrent tasks}***$$

MapReduce Model



Key observation: all k tasks that run concurrently in a node compete for CPU and I/O resources => **queuing has to be considered**

MapReduce Model

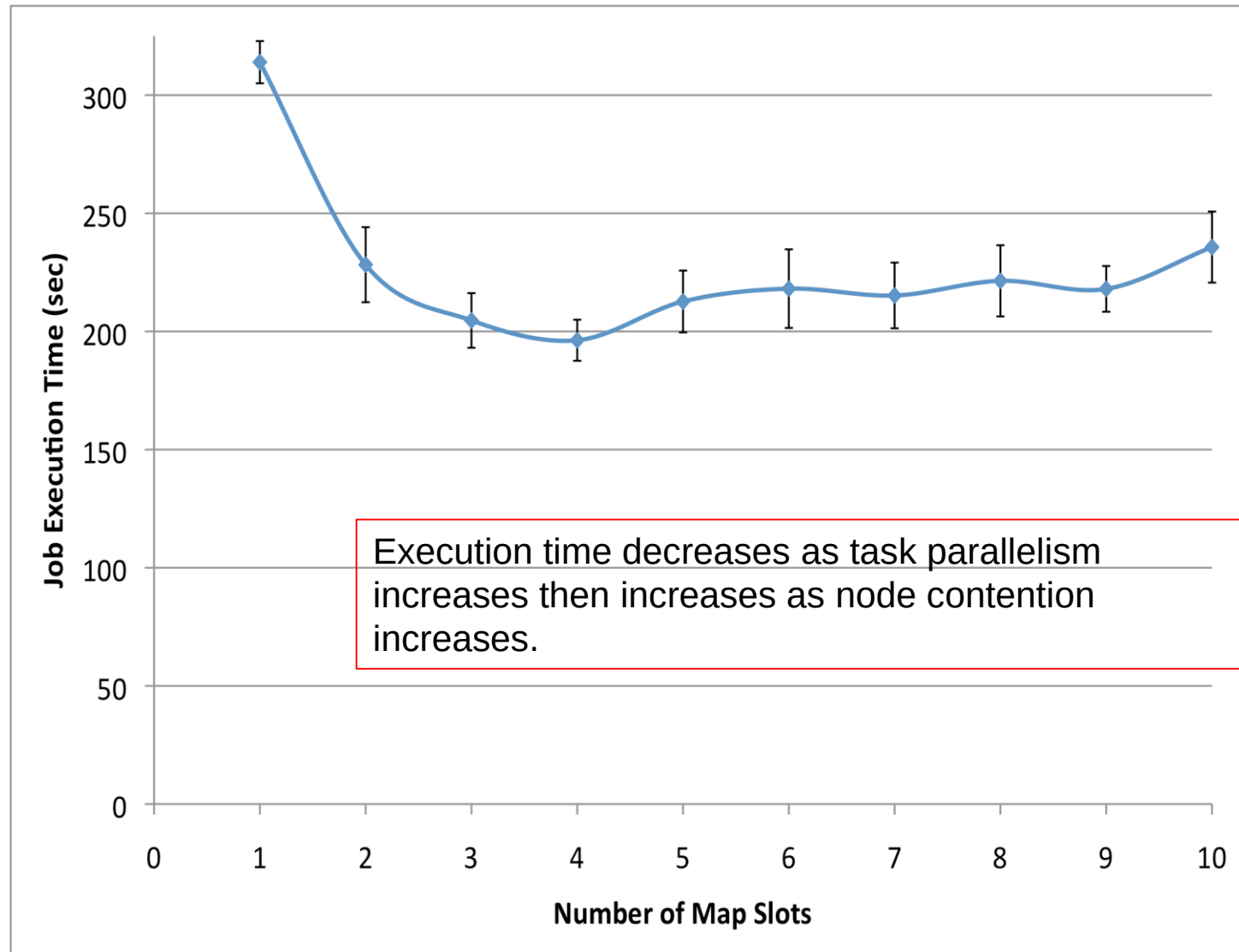
$$n' = \text{Mod}(M, n) = M - \lfloor M / n \rfloor \times n$$

- Example:
- A job has 54 map tasks and there are 4 compute nodes, each with 2 slots.
- $M = 54$; $c = 4$; $k = 2$; $n = 8$
- $n' = \text{Mod}(54, 8) = 6$

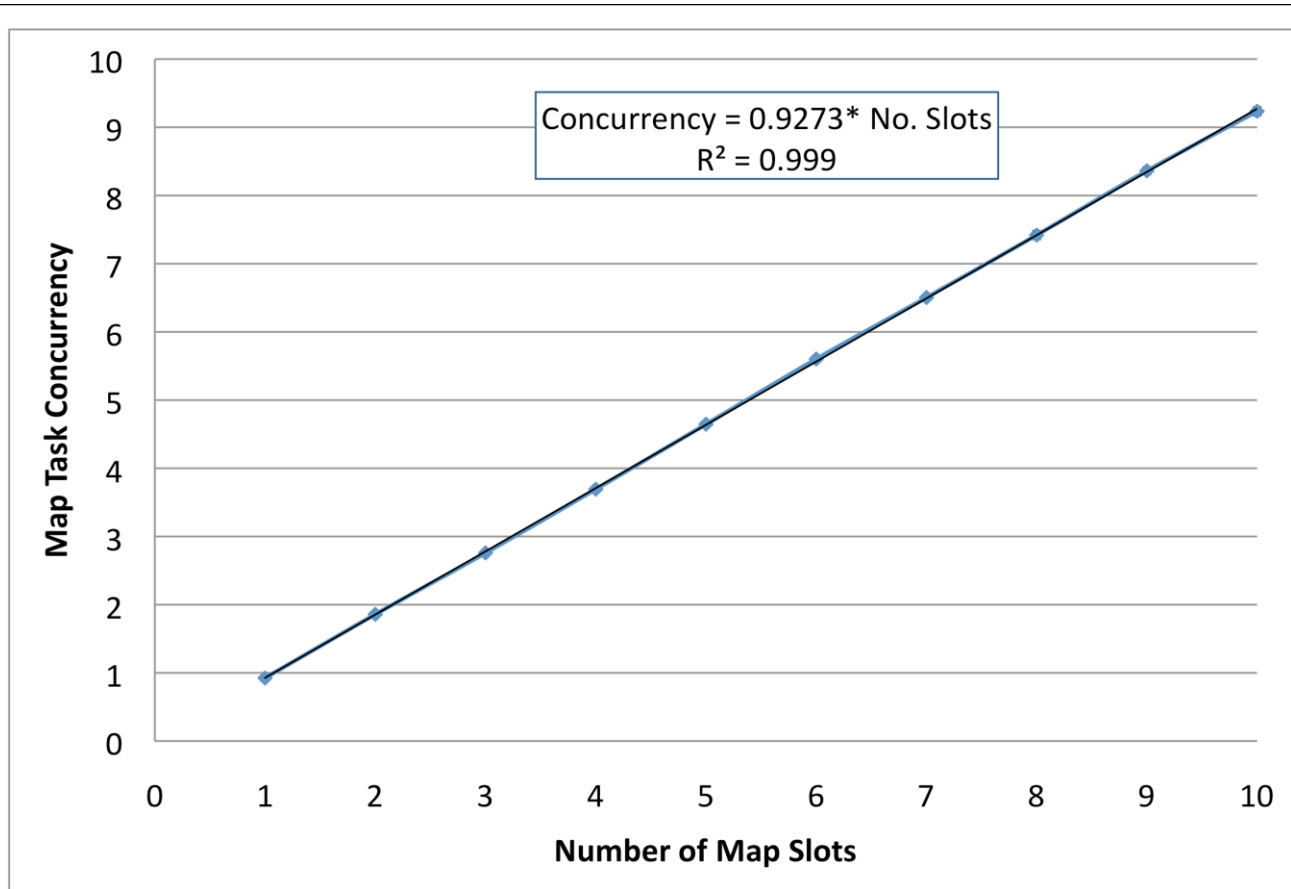
Experimental Parameters

- Size of input file: 5 MB
- Number of map slots set by varying the input split size between 100K and 600K:
 - E.g., Split size of 200 KB => 26 map slots
- No reduce tasks

Avg. execution time vs. no. slots for 54 map tasks: experiments

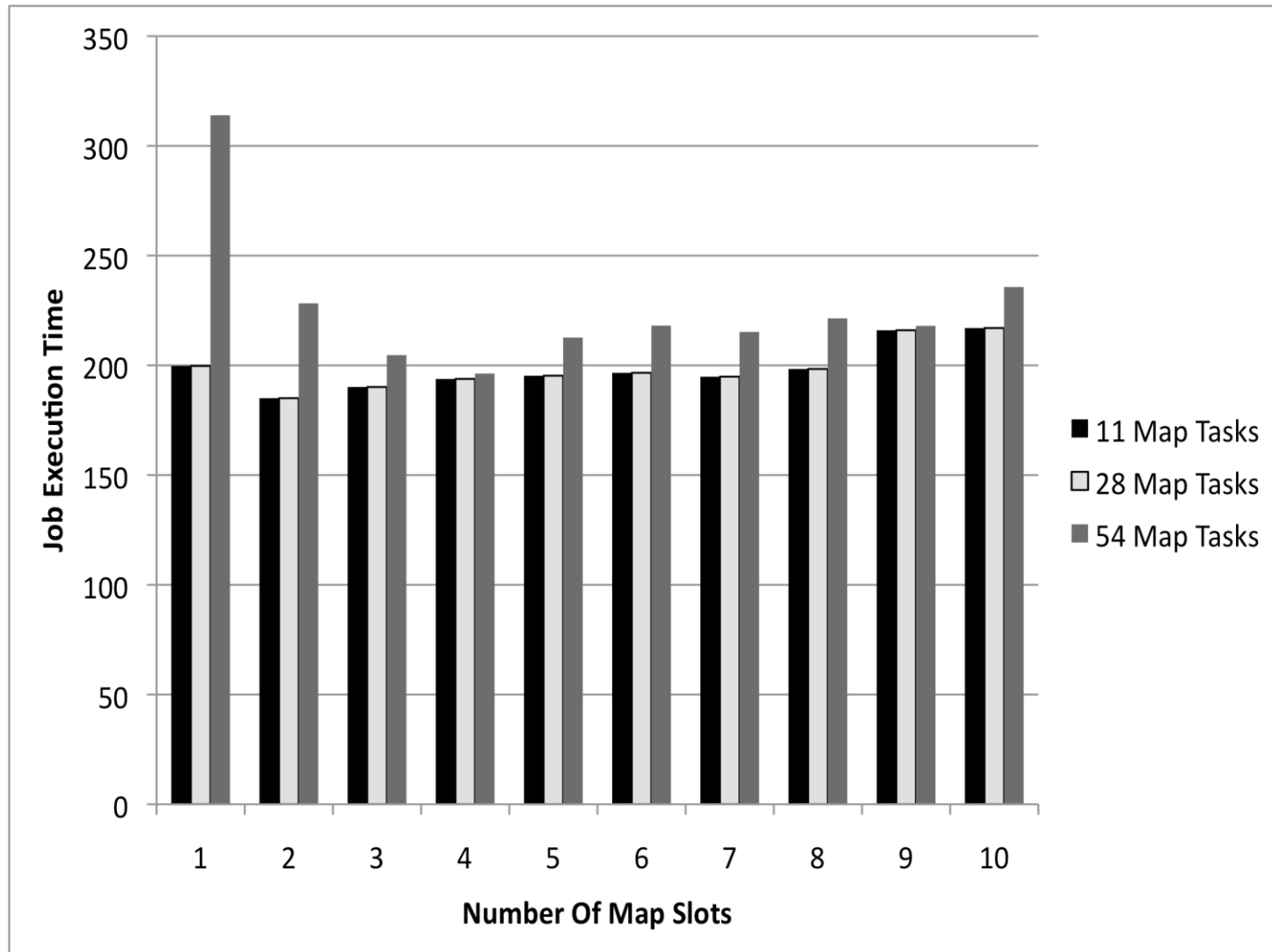


Concurrency Results

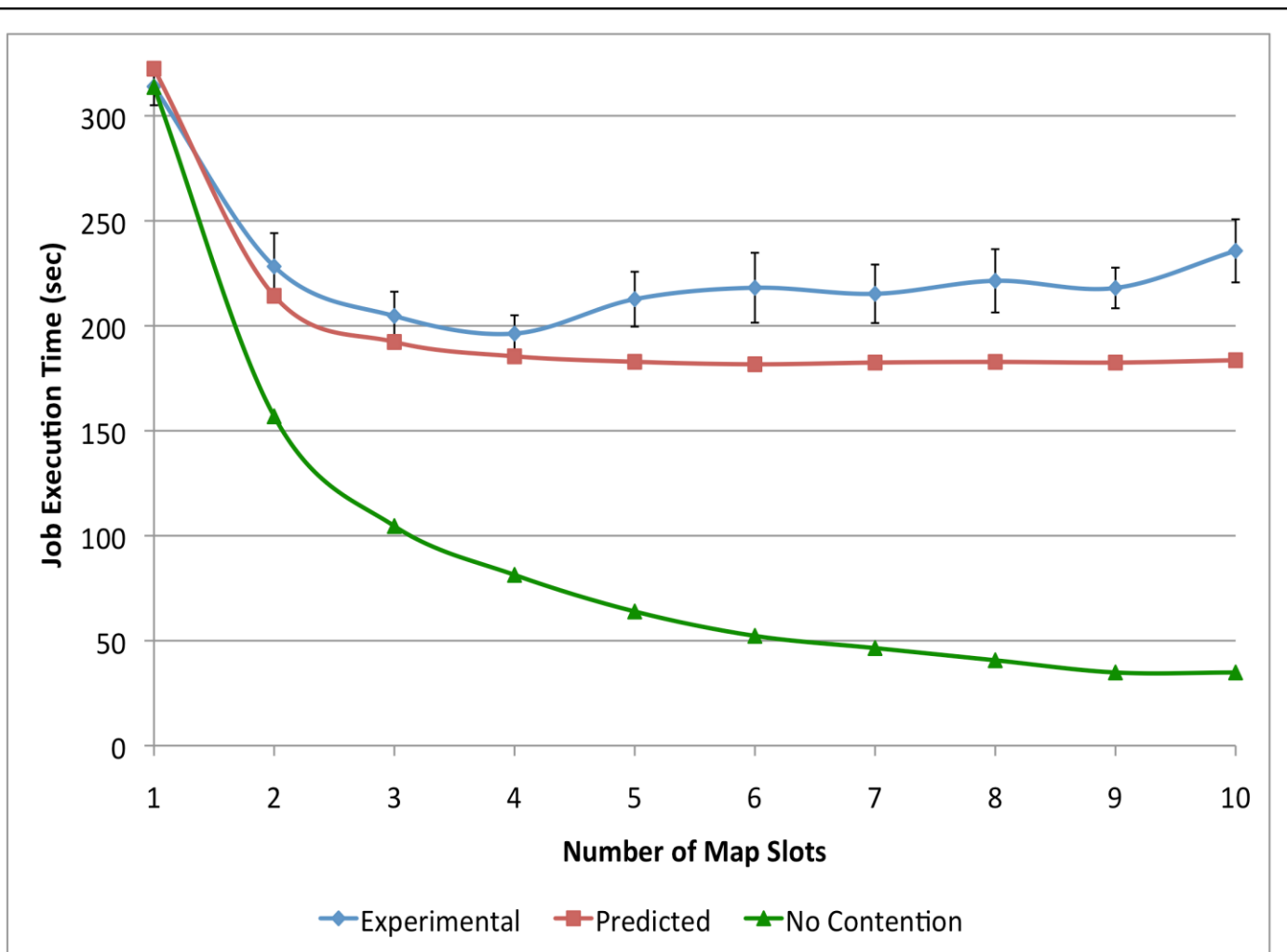


Concurrency slightly lower than number of slots due to time TaskTracker takes to detect end of a task and start a new one.

Execution time vs. no. slots for different map tasks



Execution time vs. no. slots for 54 map tasks: experiment, contention and no-contention models



Computer Architecture of WSC

- WSC often use a hierarchy of networks for interconnection
- Each 19" rack holds 48 1U servers connected to a rack switch
- Rack switches are uplinked to switch higher in hierarchy
 - Uplink has $48 / n$ times lower bandwidth, where $n = \#$ of uplink ports
 - “Oversubscription”
 - Goal is to maximize locality of communication relative to the rack



Storage

- **Storage options:**
 - Use disks inside the servers, or
 - Network attached storage through Infiniband
 - WSCs generally rely on local disks
 - Google File System (GFS) uses local disks and maintains at least three replicas

Array Switch

- **Switch that connects an array of racks**
 - Array switch should have 10 X the bisection bandwidth of a rack switch
 - Cost of n -port switch grows as n^2
 - Often utilize content addressable memory chips and FPGAs to support high-speed packet inspection.

WSC Memory Hierarchy

- Servers can access DRAM and disks on other servers using a NUMA-style interface**

	Local	Rack	Array
DRAM latency (microseconds)	0.1	100	300
Disk latency (microseconds)	10,000	11,000	12,000
DRAM bandwidth (MB/sec)	20,000	100	10
Disk bandwidth (MB/sec)	200	100	10
DRAM capacity (GB)	16	1,040	31,200
Disk capacity (GB)	2000	160,000	4,800,000

WSC Memory Hierarchy – Example I

- What is the average DRAM latency assuming that 90% of accesses are local to the server, 9% are outside the server but local to the rack , and 1% are outside the rack but within the array?
- $(90\% \times 0.1) + (9\% \times 100) + (1\% \times 300) = 12.09 \text{ msec}$

	Local	Rack	Array
DRAM latency (microseconds)	0.1	100	300
Disk latency (microseconds)	10,000	11,000	12,000
DRAM bandwidth (MB/sec)	20,000	100	10
Disk bandwidth (MB/sec)	200	100	10
DRAM capacity (GB)	16	1,040	31,200
Disk capacity (GB)	2000	160,000	4,800,000

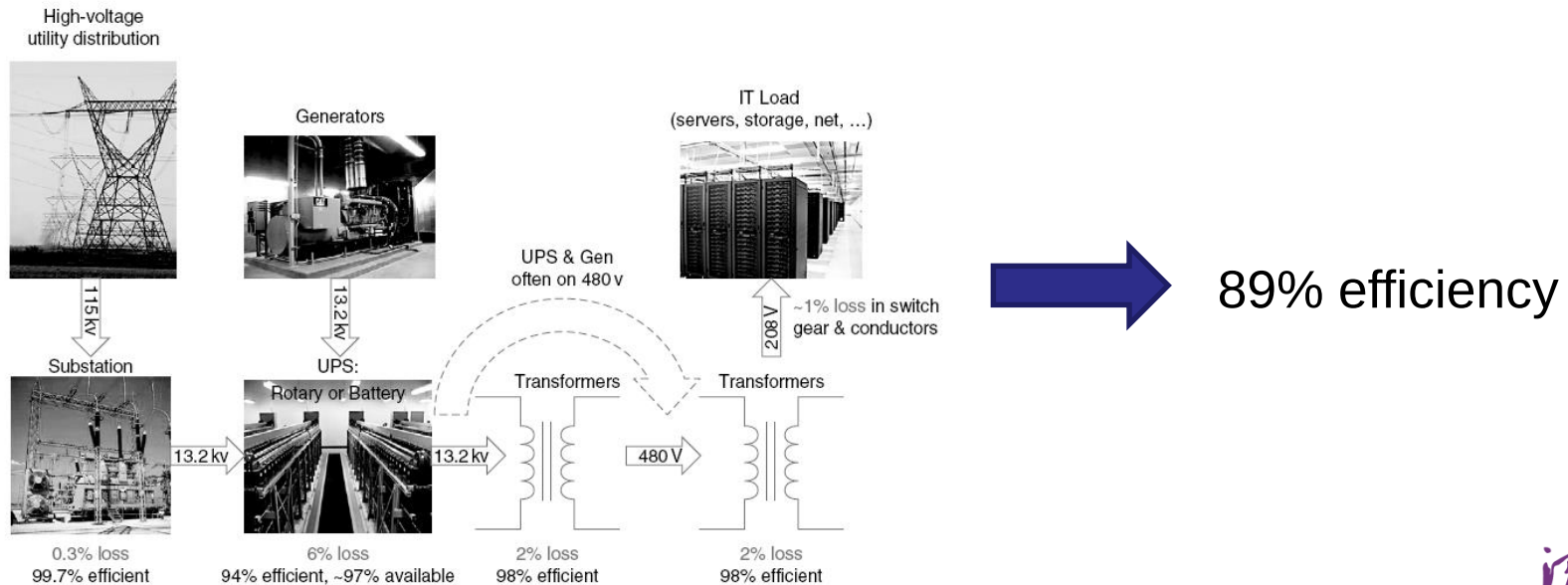
WSC Memory Hierarchy – Example II

- How long does it take to transfer 1000MB between disks within the server, between servers in the rack, and between servers in different racks of an array?
 - Within server: $1000/200=5$ sec
 - Within rack: $1000/100=10$ sec
 - Within array: $1000/10= 100$ sec

	Local	Rack	Array
DRAM latency (microseconds)	0.1	100	300
Disk latency (microseconds)	10,000	11,000	12,000
DRAM bandwidth (MB/sec)	20,000	100	10
Disk bandwidth (MB/sec)	200	100	10
DRAM capacity (GB)	16	1,040	31,200
Disk capacity (GB)	2000	160,000	4,800,000

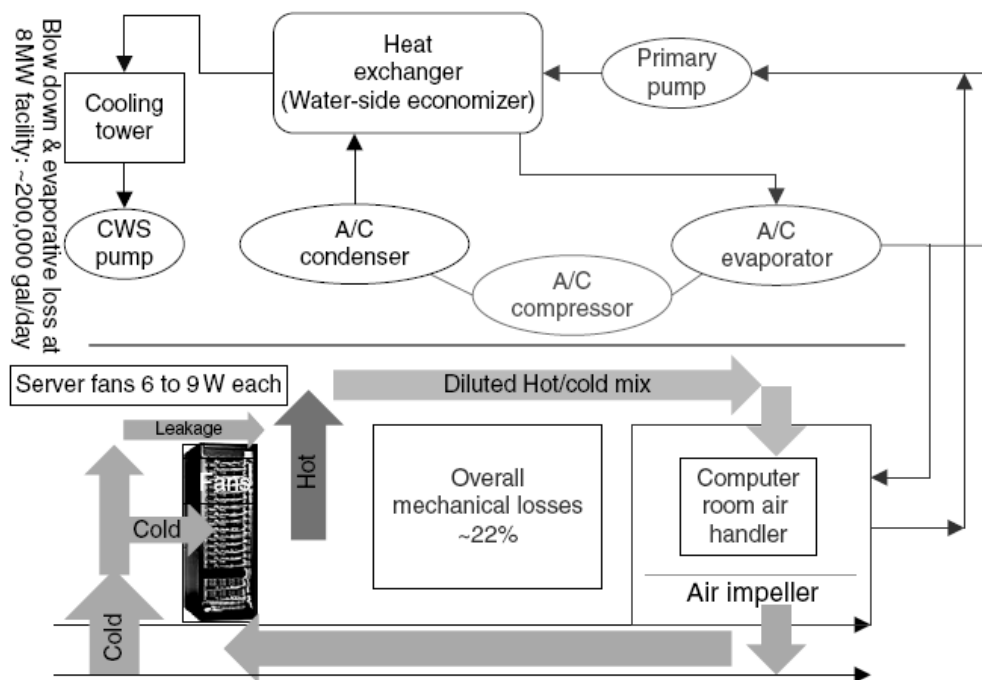
Infrastructure and Costs of WSC

- **Location of WSC**
 - Proximity to Internet backbones, electricity cost, property tax rates, low risk from earthquakes, floods, and hurricanes
- **Power distribution**



Infrastructure and Costs of WSC

- Cooling
- Air conditioning used to cool server room
- 64 F – 71 F (17-22)
 - Keep temperature higher (closer to 71 F)
- Cooling towers can also be used
 - Minimum temperature is “wet bulb temperature”



Infrastructure and Costs of WSC

- **Cooling system also uses water (evaporation and spills)**
 - E.g. 70,000 to 200,000 gallons per day for an 8 MW facility
- **Power cost breakdown:**
 - Chillers: 30-50% of the power used by the IT equipment
 - Air conditioning: 10-20% of the IT power, mostly due to fans
- **How many servers can a WSC support?**
 - Each server:
 - “Nameplate power rating” gives maximum power consumption
 - To get actual, measure power under actual workloads
 - Oversubscribe cumulative server power by 40%, but monitor power closely

Measuring Efficiency of a WSC

- **Power Utilization Effectiveness (PUE)**
 - = Total facility power / IT equipment power
 - Median PUE on 2006 study was 1.69
- **Performance**
 - Latency is important metric because it is seen by users
 - Bing study: users will use search less as response time increases
 - Service Level Objectives (SLOs)/Service Level Agreements (SLAs)
 - E.g., 99% of requests must be below 100 ms

Negative Impact of Delays at Bing

Server delay (ms)	Increased time to next click (ms)	Queries/ user	Any clicks/ user	User satisfaction	Revenue/ user
50	—	—	—	—	—
200	500	—	−0.3%	−0.4%	—
500	1200	—	−1.0%	−0.9%	−1.2%
1000	1900	−0.7%	−1.9%	−1.6%	−2.8%
2000	3100	−1.8%	−4.4%	−3.8%	−4.3%

Figure 6.12 Negative impact of delays at the Bing search server on user behavior (Schurman and Brutlag, 2009).

[Computer Architecture, Sixth Edition: A Quantitative Approach]

Cost of a WSC

- **Capital expenditures (CAPEX)**
 - Cost to build a WSC
 - Example:
 - 8 MW facility
 - 46,000 servers
 - \$88 M for facility
 - \$67M for servers
 - \$13M for networking equipment
 - PUE: 1.45
- **Operational expenditures (OPEX)**
 - Cost to operate a WSC

Cloud Computing

- **WSCs offer economies of scale that cannot be achieved with a datacenter:**
 - **5.7 times reduction in storage costs**
 - **7.1 times reduction in administrative costs**
 - **7.3 times reduction in networking costs**
 - **This has given rise to cloud services such as Amazon Web Services**
 - **“Utility Computing”**
 - **Based on using open source virtual machine and operating system software**

What is Cloud Computing?

- A modality of computing characterized by *on demand* availability of resources in a dynamic and scalable fashion.
 - resource = infrastructure, platforms, software, services, or storage.
- The *cloud provider* is responsible to make the resources available on demand to the *cloud users*.
- the cloud provider must **manage its resources** in an efficient way so that the user needs can be met when needed **at the desired QoS level**.

~~The NIST Cloud Definition Framework~~

<https://www.nist.gov/publications/nist-definition-cloud-computing>

- Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.
- This cloud model promotes availability and is composed of five essential **characteristics**, three **service models**, and four **deployment models**.

NIST Cloud Conceptual Reference Model

http://www.nist.gov/itl/cloud/upload/SP_500_293_volumell.pdf

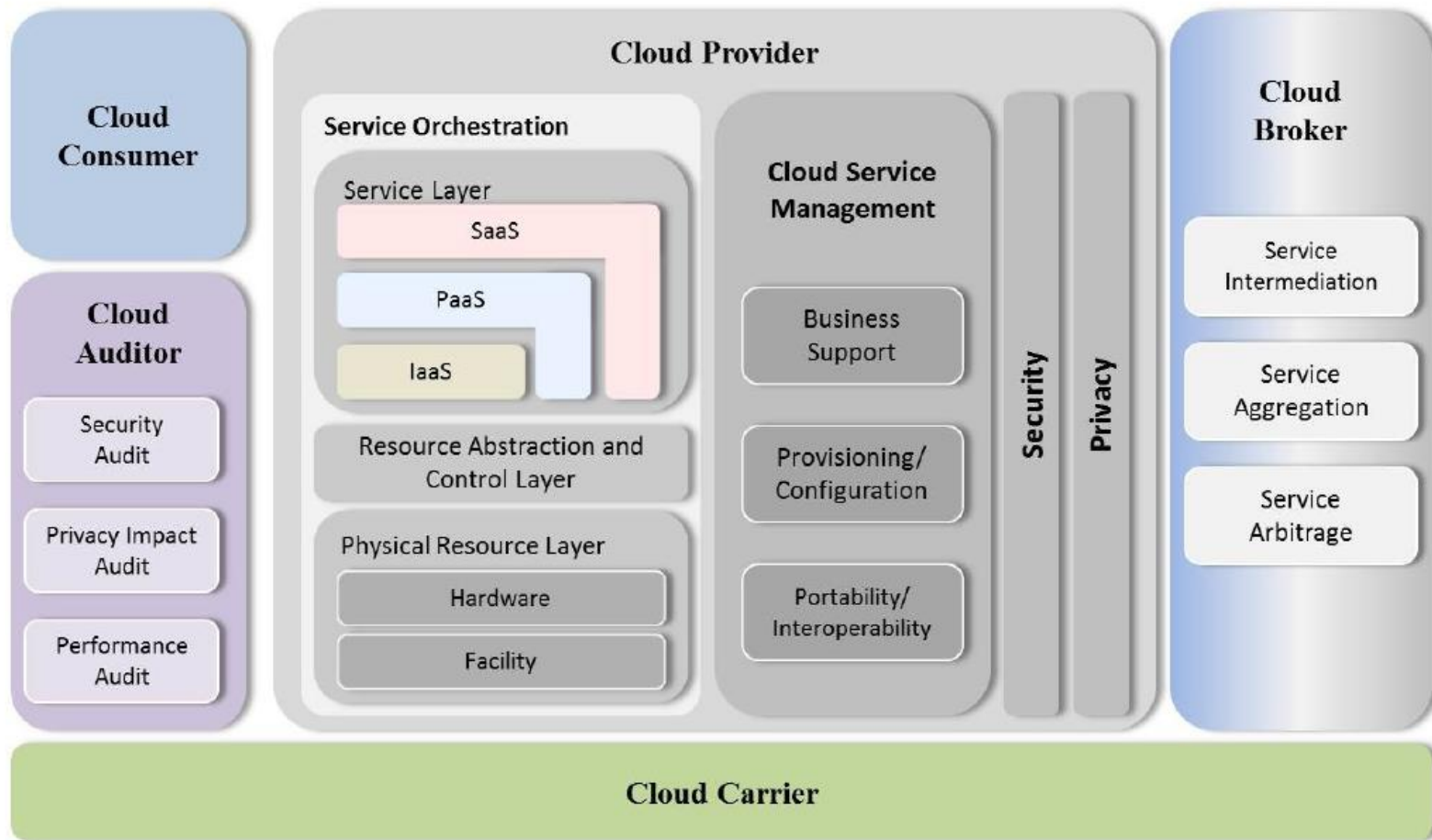


Figure 2: The Conceptual Reference Model

Is Cloud Computing Analogous to an Electric Grid?

Consumers use electric energy on-demand according to their needs and pay based on their consumption.

Power utilities have to be able to dynamically determine how to match demand and supply.

The product delivered by the power grid is homogeneous (e.g., 110 V of alternating current at 60 Hz).

One can plug any appliance to the power grid and it will work seamlessly as long as it conforms to a very simple specification of voltage and frequency.

Cloud computing users use resources on demand according to their needs and pay based on their consumption.

Cloud computing providers have to be able to dynamically determine how to match demand and supply.

Clouds offer a variety of resources on demand.

The APIs offered by cloud providers are not standardized and may be very complicated in many cases.

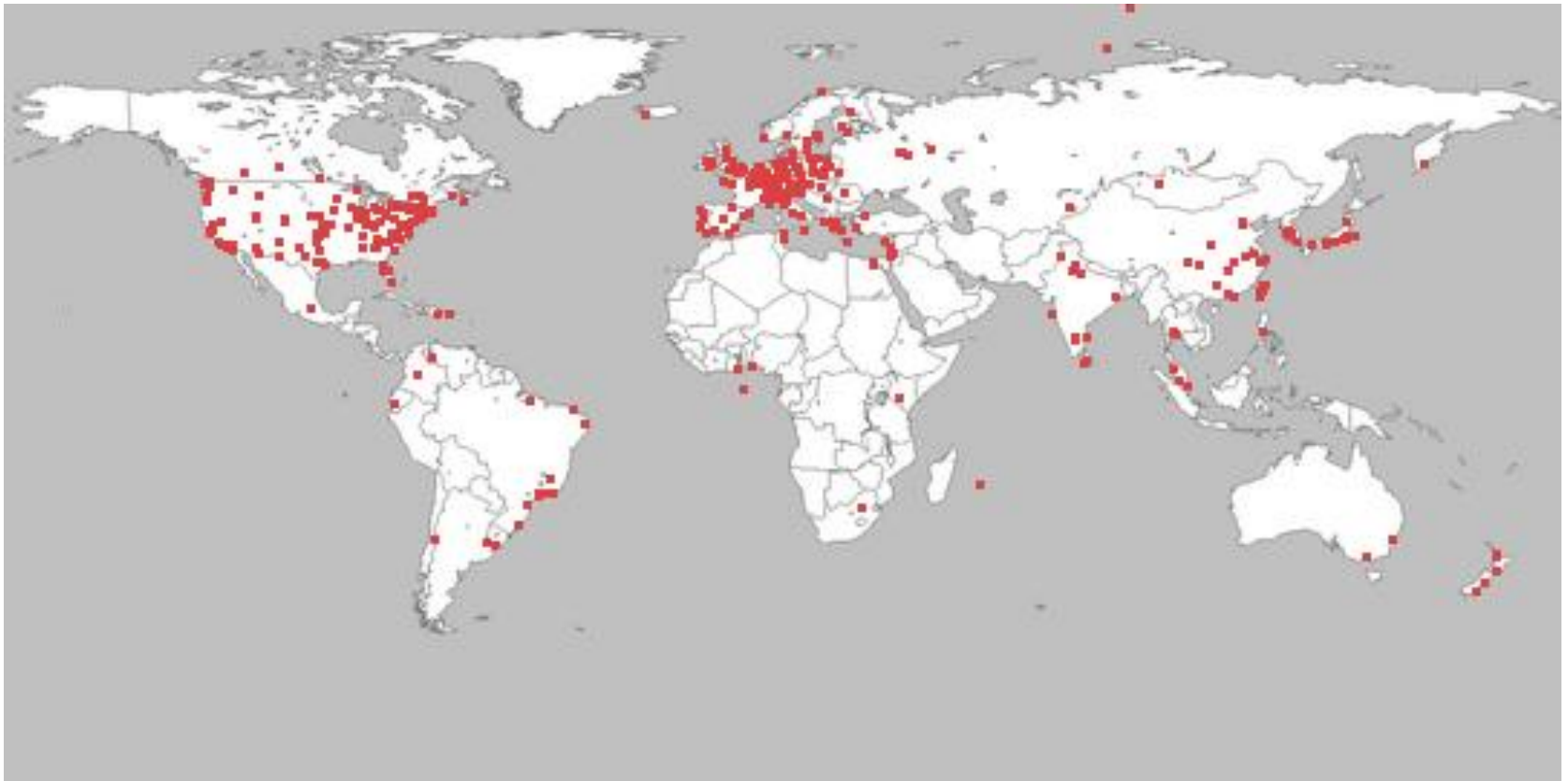
Advantages of Cloud Computing

- *Pay as you go.*
- *No need to provision for peak loads.*
- *Time to market.*
- *Consistent performance and availability.*

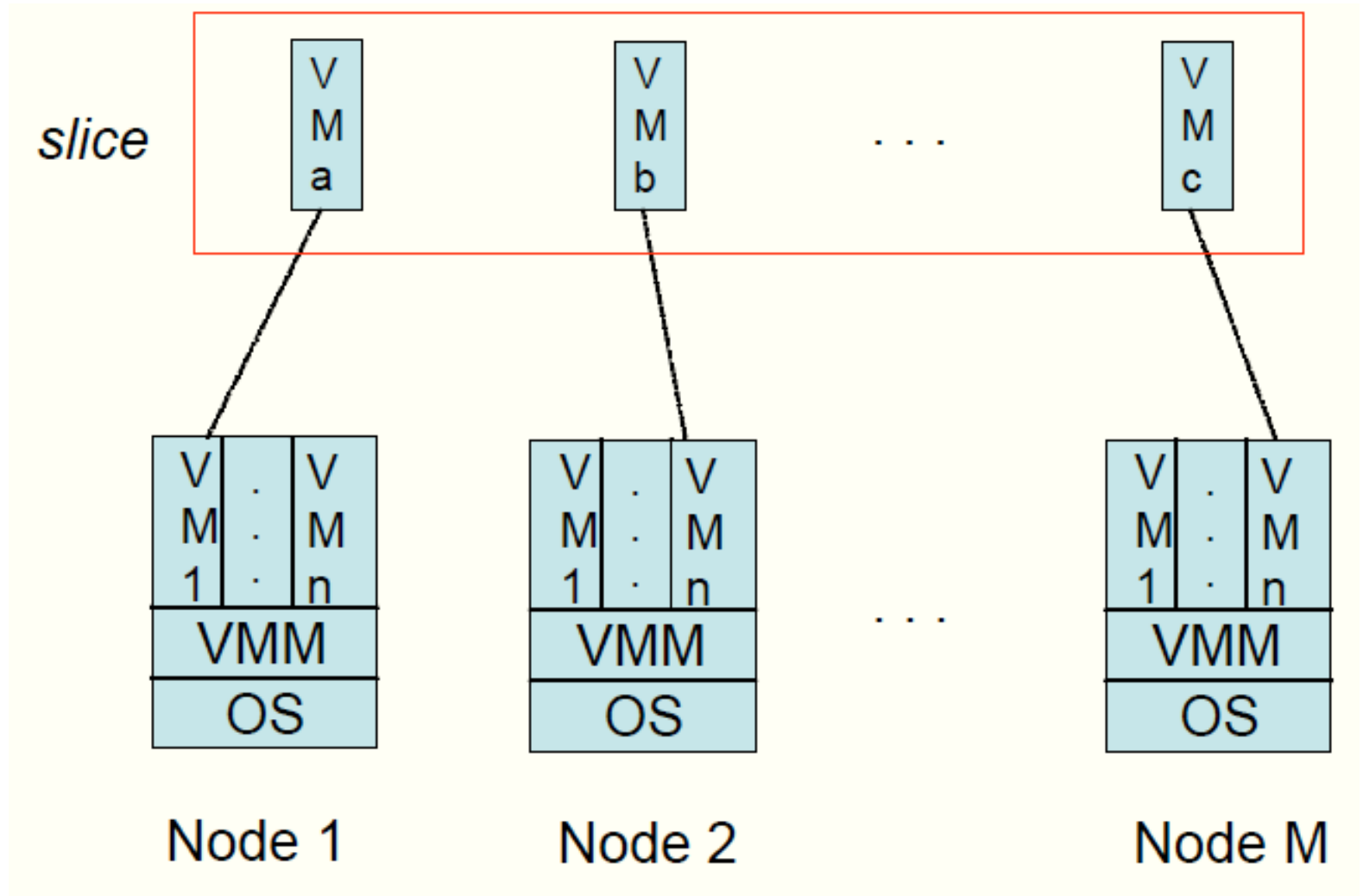
Potential Drawbacks of Cloud Computing

- *Privacy and security.*
- *External dependency for mission critical applications.*
- *Disaster recovery.*
- *Monitoring and Enforcement of SLAs.*

Example: PlanetLab - www.planet-lab.org



Example: PlanetLab - www.planet-lab.org



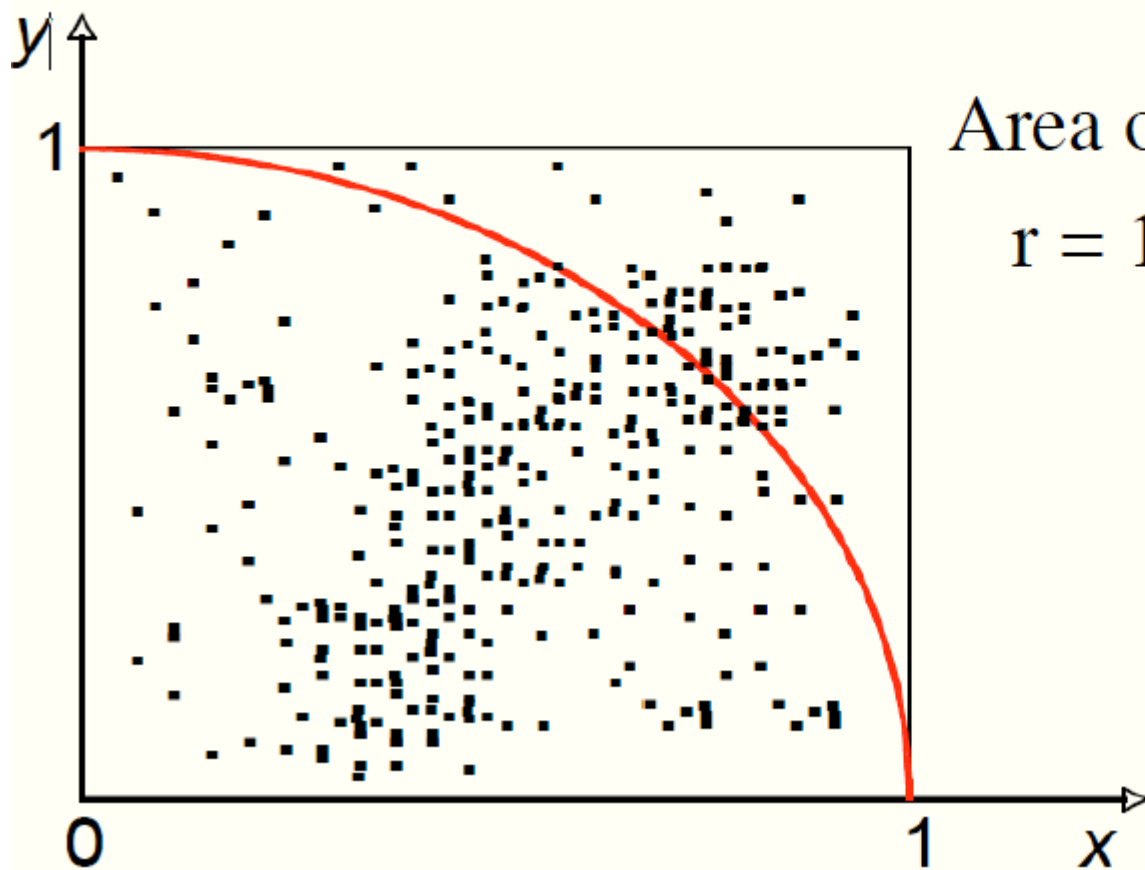
Examples: PlanetLab

- 717 sites with 1353 nodes scattered over 48 countries at its peak.
- Shared resources include CPU cycles, storage, and memory.
- PlanetLab Central API allow users to create automated scripts to monitor node availability.

See https://planetlab.cs.princeton.edu/doc/plc_api.html

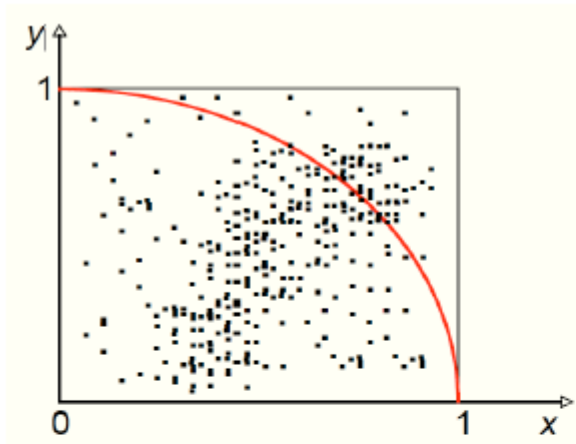
Parallel Computation of π Using Planetlab

Area of a circle: πr^2



Area of a quadrant when
 $r = 1: \pi/4$

Parallel Computation of π Using Planetlab



Area of a circle: πr^2

Area of a quadrant when

$$r = 1: \pi/4$$

Area of a quadrant $\approx$ number of random points in quadrant divided by total number of random points.

Node 1

...

Node n

m/n points

m/n points

Parallel Computation of π Using Planetlab

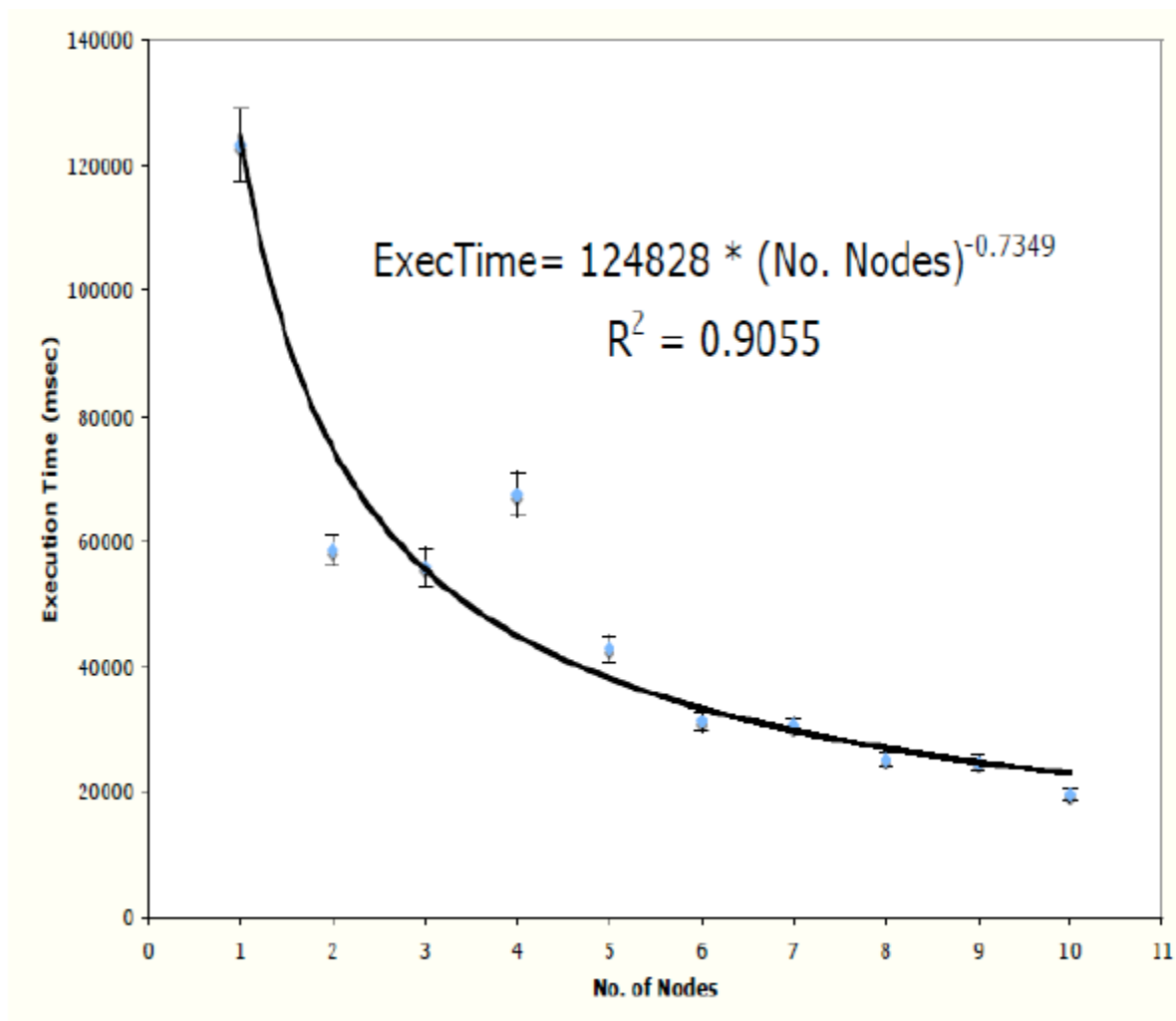
1. (At the master node). Send m and n to all n slave nodes.
2. (At each slave node $i, i = 1, \dots, n$).
 $\text{NumPointsInQuadrant}_i \leftarrow 0$.
3. (At each slave node $i, i = 1, \dots, n$). Repeat (a) and (b) m/n times.
 - (a) Randomly select a point (x, y) such that x and y are random numbers uniformly distributed in $[0, 1]$.
 - (b) If $\sqrt{x^2 + y^2} \leq 1$ then
 $\text{NumPointsInQuadrant}_i \leftarrow$
 $\text{NumPointsInQuadrant}_i + 1$.
4. (At each slave node $i, i = 1, \dots, n$).
Send $\text{NumPointsInQuadrant}_i$ to the master node.
5. (At the master node).
 $\pi \leftarrow 4 \times \sum_{i=1}^n \text{NumPointsInQuadrant}_i / m$.

Parallel Computation of π Using Planetlab

Node Name	Location	Participation
UP1	Univ. Pennsylvania	1-10
UP2	Univ. Pennsylvania	2-10
GT1	Georgetown University	3-10
GT2	Georgetown University	4-10
GMU3	George Mason University	5-10
GMU4	George Mason University	6-10
CT1	Caltech	7-10
CT2	Caltech	8-10
CN4	Cornell University	9-10
VT	Virginia Tech	10

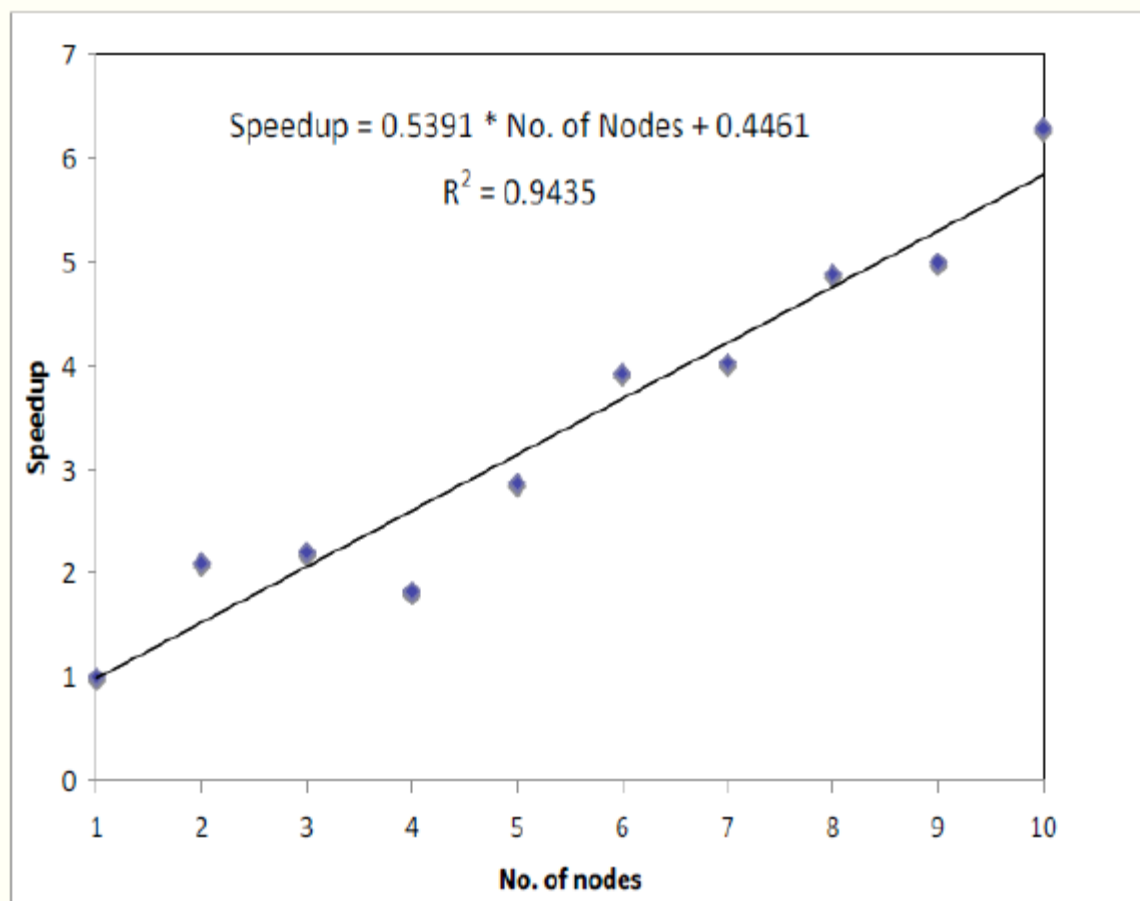
All nodes: 2 Intel Core 2 Duo E6550 Processor
@ 2.44 GHz with 3.44GB memory.

Parallel Computation of π Using Planetlab



$m = 1$ billion

Parallel Computation of π Using Planetlab



$$S = \frac{E_1}{E_n}$$

$m = 1$ billion

Example: Amazon's Elastic Computing Cloud (EC2)

- <http://aws.amazon.com/ec2/>
- Virtual site farm
- Users request the number and type of
- compute *instances* they need:
 - Standard
 - High-memory
 - High-CPU.
- Payment: by instance-hour
- One EC2 compute unit provides the equivalent of the CPU capacity of a 1.0-1.2 GHz 2007 Opteron or 2007 Xeon processor.

Example: Amazon's Elastic Computing Cloud (EC2)

- EC2's Auto Scaling allows users to determine when to scale up or down their EC2 usage according to user-defined conditions.
 - Saves money
- EC2's CloudWatch aggregates and reports metrics for CPU utilization, data transfer, and disk usage and activity for each EC2 instance.

Amazon's EC2 Compute Instances

First Generation

First generation (M1) Standard instances provide customers with a balanced set of resources and a low cost platform that is well suited for a wide variety of applications.

M1 Small Instance – default*

- 1.7 GiB memory
- 1 EC2 Compute Unit (1 virtual core with 1 EC2 Compute Unit)
- 160 GB instance storage
- 32-bit or 64-bit platform
- I/O Performance: Moderate
- EBS-Optimized Available: No
- API name: m1.small

M1 Medium Instance

- 3.75 GiB memory
- 2 EC2 Compute Unit (1 virtual core with 2 EC2 Compute Unit)
- 410 GB instance storage
- 32-bit or 64-bit platform
- I/O Performance: Moderate
- EBS-Optimized Available: No
- API name: m1.medium

M1 Large Instance

- 7.5 GiB memory
- 4 EC2 Compute Units (2 virtual cores with 2 EC2 Compute Units each)
- 850 GB instance storage
- 64-bit platform
- I/O Performance: Moderate
- EBS-Optimized Available: 500 Mbps
- API name: m1.large

M1 Extra Large Instance

- 15 GiB memory
- 8 EC2 Compute Units (4 virtual cores with 2 EC2 Compute Units each)
- 1,690 GB instance storage
- 64-bit platform
- I/O Performance: High
- EBS-Optimized Available: 1000 Mbps
- API name: m1.xlarge

Amazon's EC2 Pricing

Instance	Per hour	Ratio to small	Compute units	Virtual cores	Compute units/core	Memory (GB)	Disk (GB)	Address size
Micro	\$0.020	0.5–2.0	0.5–2.0	1	0.5–2.0	0.6	EBS	32/64 bit
Standard Small	\$0.085	1.0	1.0	1	1.00	1.7	160	32 bit
Standard Large	\$0.340	4.0	4.0	2	2.00	7.5	850	64 bit
Standard Extra Large	\$0.680	8.0	8.0	4	2.00	15.0	1690	64 bit
High-Memory Extra Large	\$0.500	5.9	6.5	2	3.25	17.1	420	64 bit
High-Memory Double Extra Large	\$1.000	11.8	13.0	4	3.25	34.2	850	64 bit
High-Memory Quadruple Extra Large	\$2.000	23.5	26.0	8	3.25	68.4	1690	64 bit
High-CPU Medium	\$0.170	2.0	5.0	2	2.50	1.7	350	32 bit
High-CPU Extra Large	\$0.680	8.0	20.0	8	2.50	7.0	1690	64 bit
Cluster Quadruple Extra Large	\$1.600	18.8	33.5	8	4.20	23.0	1690	64 bit

Figure 6.15 Price and characteristics of on-demand EC2 instances in the United States in the Virginia region in January 2011. Micro Instances are the newest and cheapest category, and they offer short bursts of up to 2.0

Examples: Google's App Engine

- Web applications can be deployed on Google's infrastructures.
- Applications can run in Java or Python run-time environments.
- Free startup: all applications can use up to 500MB of storage and enough CPU and bandwidth to support an efficient app serving around 5 million page views a month for free.
 - After that, pay according to resource usage.

Examples: Google's App Engine

- App Engine provides a powerful distributed data storage service that features a query engine and transactions.
- Applications may include:
 - dynamic web serving
 - persistent storage
 - automatic scaling and load balancing
 - user authentication
 - task queues
 - scheduled tasks

Capacity Planning for the Cloud: from the consumer's point of view

- Problems for consumers:
 - How to select SLAs for various QoS metrics in a way that maximizes a utility function for the consumer subject to cost-constraints?

Capacity Planning for the Cloud:

from the consumer's point of view

- SLA_r : SLA (in seconds) on the average response time per transaction.
- SLA_x : SLA (in tps) on the transaction throughput.
- SLA_a : SLA on the cloud availability.
- $C_r(SLA_r)$: per transaction cost (in cents) when the negotiated response time SLA is SLA_r .
- $C_x(SLA_x)$: per transaction cost (in cents) when the negotiated throughput SLA is SLA_x .
- $C_a(SLA_a)$: per transaction cost (in cents) when the negotiated availability SLA is SLA_a .

Capacity Planning for the Cloud:

from the consumer's point of view

- U : global utility. The global utility function is composed of terms that represent the utility for various metrics such as response time, throughput, and availability. The utility is a dimensionless number in the $[0,1]$ range.
- w_r, w_x, w_a : weights associated to response time, throughput, and availability, respectively, used to compute the global utility. $w_r + w_x + w_a = 1$.

Capacity Planning for the Cloud: from the consumer's point of view

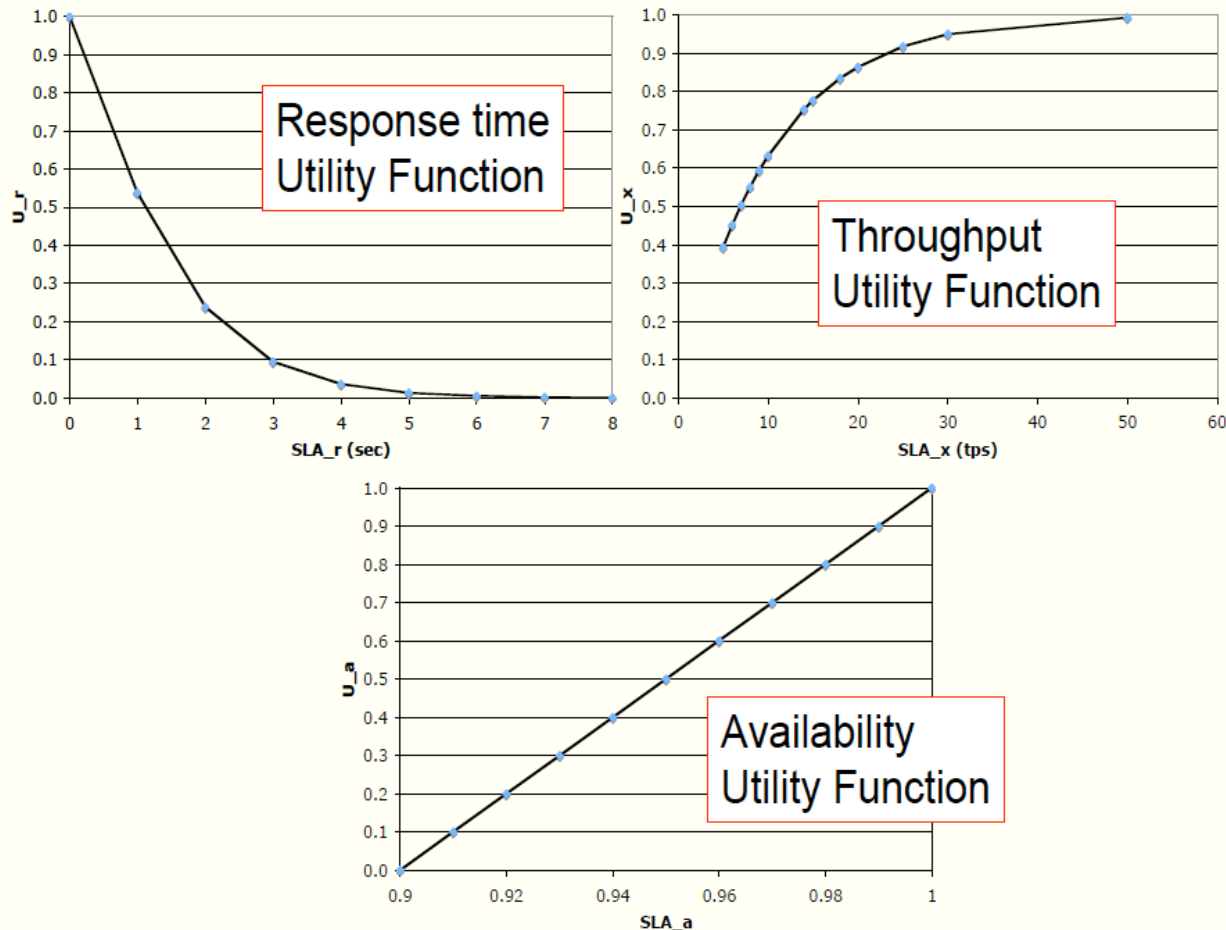
$$C_r(\text{SLA}_r) = \alpha_r e^{-\beta_r \text{SLA}_r}$$

$$C_x(\text{SLA}_x) = \alpha_x \text{SLA}_x$$

$$C_a(\text{SLA}_a) = e^{\beta_a \text{SLA}_a} - e^{0.9\beta_a}, \text{SLA}_a \geq 0.9.$$

$$U = w_r \frac{2.0 e^{-\text{SLA}_r}}{1 + e^{-\text{SLA}_r}} + w_x(1 - e^{-0.1 \text{SLA}_x}) + w_a(10 \text{SLA}_a - 9).$$

Capacity Planning for the Cloud: from the consumer's point of view



Capacity Planning for the Cloud: from the consumer's point of view

$$\text{maximize } U = f(\text{SLA}_r, \text{SLA}_x, \text{SLA}_a)$$

subject to

$$\gamma_r^{\min} \leq \text{SLA}_r \leq \gamma_r^{\max}$$

$$\gamma_x^{\min} \leq \text{SLA}_x \leq \gamma_x^{\max}$$

$$\gamma_a^{\min} \leq \text{SLA}_a \leq \gamma_a^{\max}$$

$$C_r(\text{SLA}_r) + C_x(\text{SLA}_x) + C_a(\text{SLA}_a) \leq C_{\max}$$

Capacity Planning for the Cloud: from the consumer's point of view

$$\text{maximize } U = f(\text{SLA}_r, \text{SLA}_x, \text{SLA}_a)$$

subject to

$$\gamma_r^{\min} \leq \text{SLA}_r \leq \gamma_r^{\max}$$

$$\gamma_x^{\min} \leq \text{SLA}_x \leq \gamma_x^{\max}$$

$$\gamma_a^{\min} \leq \text{SLA}_a \leq \gamma_a^{\max}$$

$$C_r(\text{SLA}_r) + C_x(\text{SLA}_x) + C_a(\text{SLA}_a) \leq C_{\max}$$

69

Solvers:

- **NEOS:** <http://neos.mcs.anl.gov>
- **MS Excel's Solver** (see Tools menu)

Capacity Planning for the Cloud:

from the consumer's point of view

C_{max}	Utility	SLA_r	SLA_x	SLA_a
0.70	0.641	1.000	5.625	0.999
0.60	0.438	3.543	5.000	0.999
0.55	0.366	4.000	5.000	0.978
0.50	0.279	4.000	5.000	0.949

$$w_r = 0.4; w_x = 0.3; w_a = 0.3$$

Capacity Planning for the Cloud: from the provider's point of view

- Providers have to deal with:
 - Large and complex infrastructures
 - Hard to predict and time-varying workloads
- Providers need to implement autonomic computing techniques that are capable to dynamically shift resources without human intervention to cope with negotiated SLAs.

Acknowledgement

- The slides are modified according to the following materials
 - Slides from Computer Architecture QA 5rd
 - Slides from George Mason University